

task_bridge

コンテンツ

タスクボードと信頼できる唯一の情報源を、完璧に双方向同期。

概要

task_bridgeは、Go内に実装された汎用的で分離型の双方向タスク／ボード同期エンジンです。プロジェクトの作業可能アイテム (SQLite) を信頼できる唯一の情報源として、トラッカー文書およびリモートボード（第一ターゲット：ClickUp、Jira／Linearは計画中）との間で、決定論的な「最終編集優先」、ドライラン優先、データ破損なしのセマンティクスを用いて同期します。

短い説明

プロジェクトに依存しないGoのサブモジュールで、SQLiteの作業可能アイテム (SSoT) ↔ トラッカー文書 ↔ リモートボード (ClickUpを最初にサポート) を双方向同期します。決定論的な「最終編集優先」、ドライラン優先、HMAC検証済みウェブフックを採用。すべての認証情報とIDは、実行時に利用者側から注入されます。

長い説明

どのチームも、やがて同じ作業の二つの台帳を持つようになります。一つは実際の作業を反映したもの——コード、文書、内部データベース——もう一つはマネージャーが監視するボード (ClickUpなど) です。どちらか一方に手を加えた瞬間から、二つの台帳は乖離し始め、手作業での調整はまさに面倒でミスが起こりやすい作業となり、誰も確実に行わなくなります。task_bridgeは、このギャップを解消するために設計されました。プロジェクトの作業可能アイテム (SQLite) を信頼できる唯一の情報源とし、そのトラッカー文書とリモートボード (最初にサポートするのはClickUp、将来的にはJiraやLinearも対象) を、一つのシステムとして連動させます。同期は決定論的 (「最終編集優先」)、ドライラン優先で、絶対に譲れない約束が一つあります。それは、データを破損させたり失っ

たりせず、片方の情報が古くなったまま放置されることもないというものです。不注意な同期で一週間分の作業が上書きされるような状況では、この安全性こそがすべてです。

アーキテクチャ的には、他のプロジェクトに利用される厳密なサブモジュールであり、憲章の分離契約 (§11.4.28) に基づき、完全にプロジェクトに依存しません。プロジェクト固有の値は一切含まず、すべての認証情報、ボード／フォルダーID、アイテムキー項目、データベースパスは、利用者が実行時にpkg/config.Configを通じて注入します。モジュールは明確に階層化されており、CLI (reconcile/push/pull/resolve/status/conflicts/init) と、長時間稼働するデーモン（ウェブフック受信＋定期的な調整）で構成されます。また、MITライセンスのraksul/go-clickupをラップした薄いクライアント、ボード／フォルダーURLをライブのAPIプロンプでIDに変換するリゾルバー（URLの文法推測は行わない）、ローカルの作業可能アイテムとリモートタスクフィールドをマッピングするコンポーネント、明示的な競合解決を伴う「最終編集優先」同期エンジン、そしてX-SignatureのHMAC-SHA256検証を行うウェブフック受信機能を備えています。

成熟度については正直に述べると、これはP1の骨格です。レイアウト、インターフェース、エントリーポイント、分離境界は整備されていますが、同期ロジックとClickUpへのライブコールはまだ実装されていません（すべてのスタブは「未実装」エラーを明示的に返す、フェイクなしのルールに従っています）。

なぜ開発したのか

チームは「実際の」作業状態をコードや文書に保持し、マネージャーはClickUpのようなボードで管理しますが、両者は常に乖離していきます。task_bridgeは、これらを一つのシステムとして扱い、決定論的かつ安全に同期することで、どちらの情報も古くなったり不正確になったりしないようにします。

コンテンツ

なぜゲームチェンジャーなのか

双方向のボード同期は通常、一度限りの固定的な統合であり、どのチームも不完全な形で再構築している。task_bridgeはこれを、再利用可能でクレデンシャル注入型のライブラリとして再定義し、データ安全性の保証を最初から組み込んだ——ドライラン優先、決定論的な最終編集優先、HMAC検証済みイベント——これにより、どのプロジェクトも脆弱なコネクタを再び書くことなく、設定を注入するだけで信頼性の高いボード統合を採用できる。

革新的なポイント

- 三方向双方向同期: SQLite SSoT ↔ トラッカー文書 ↔ リモートボード
- 完全な疎結合 (§11.4.28) : プロジェクト固有の値は一切なし、すべて実行時に注入
- Live-API URL→ID解決による、脆弱なURL文法解析の排除
- HMAC-SHA256検証済みウェブフックによるリアルタイムイベント取り込み

課題と解決策

- 三つのソース間でのデータ安全性: 決定論的な最終編集優先、ドライラン優先、明示的な競合結果により解決
- 結合なしでの再利用性: pkg/config注入境界 (プロジェクト固有の情報は含まず) により解決
- 信頼性の高いボード識別: Live APIプロブによるURLからIDへの解決で解決
- 正直な足場: 未実装のスタブは明示的な「未実装」エラーを返す (偽装なし) ことで解決

技術スタック (理由と方法)

- **Go** — エンジン、CLI (cmd/task_bridge) 、およびデーモン (cmd/task_bridged)
- **SQLite** — ワーク可能アイテムの単一情報源
- **raksul/go-clickup (MIT)** — ClickUpトランスポートラッパー
- **HMAC-SHA256** — ウェブフック署名検証
- **cron + ウェブフック** — デーモン調整 + リアルタイムイベント取り込み
- **pkg/config** — 実行時クレデンシャル/ID注入境界

ステータスの正直さ: これは**P1**足場段階であり、同期ロジックはまだ実装されていない。出荷済みとして提示しないこと。